

# Homework 1

IE 523: Financial Computing

University of Illinois Urbana-Champaign

Fall 2026

Instructor: Abhishek Umrawal

Maximum Points: 100

Due Date: Wednesday, September 2, 2026, 11:59 PM

## Instructions

1. Submit all source files (`.h/.hpp`, `.cpp`, and `main.cpp`).
2. Use object-oriented programming and build on the code developed during lectures.
3. Show clear organization of your code using separate header, source, and main files.
4. Include sufficient comments explaining the key parts of your implementation.
5. You may use any software to create plots. Include the plot in your submission.
6. You may use  $\text{\LaTeX}$ , R Markdown, Markdown, Word, or PDF for your report. Handwritten submissions are acceptable; however, professional typesetting is strongly encouraged, especially for students interested in quantitative finance, industry research, or graduate study.
7. Submit a ZIP file containing all source code files, plots, and your report.
8. Submit your work through Gradescope.

## Background

During lectures, we developed a simple `Stock` class and used it to read a portfolio from a file, calculate stock values, and compute the total value of a portfolio. For this assignment, you will extend that code and develop a simple portfolio management simulator. The goal is to practice object-oriented programming in C++ while exploring a basic trading strategy.

## Problems

### 1. Portfolio Simulation and Trading Strategy (100 points)

Consider the portfolio used during lectures:

```
NVDA 110 78.54
AAPL 200 189.72
GOOG 100 173.88
```

The columns represent:

- (a) Stock ticker
- (b) Number of shares
- (c) Initial stock price

#### Part A: Extend the Stock Class

Extend the `Stock` class developed during lectures. Your class should support at least the following functionality:

- Compute the value of a stock position.
- Update the stock price.
- Buy additional shares.
- Sell shares.

You are free to add any additional methods or data members that you believe are useful.

#### Part B: Create a Portfolio Class

Create a new class named `Portfolio` that manages a collection of `Stock` objects. At a minimum, your class should maintain:

- A collection of stocks.
- Available cash.
- Current portfolio value.

Your class should provide methods that allow you to:

- Buy shares.
- Sell shares.
- Compute total portfolio value.
- Access portfolio statistics.

#### Part C: Price Simulation

Assume the investor starts with \$100,000 in cash. You will simulate prices for  $T=1000$  time periods. At each time step, generate a random return

$$r_t = \mu + \sigma Z_t,$$

where  $Z_t \sim N(0, 1)$ .

Then update prices using

$$P_{t+1} = P_t(1 + r_t).$$

Use the following parameters:

| Stock | $\mu$  | $\sigma$ |
|-------|--------|----------|
| NVDA  | 0.0008 | 0.030    |
| AAPL  | 0.0004 | 0.015    |
| GOOG  | 0.0005 | 0.020    |

Interpretation:

- NVDA has the largest volatility.
- AAPL has the smallest volatility.
- GOOG lies between the two.

You may use the facilities provided by the C++ `<random>` library to generate normal random variables.

### Part D: Trading Strategy

For each stock, maintain a reference price  $B$ . Initially,  $B = P_0$ . At every time step:

**Buy Rule** If

$$P_t \leq 0.95B,$$

buy 10 shares of the stock, provided sufficient cash is available. After the purchase, set  $B = P_t$ .

**Sell Rule** If

$$P_t \geq 1.10B,$$

sell 10 shares of the stock, provided at least 10 shares are currently owned. After the sale, set  $B = P_t$ .

**Hold Rule** Otherwise, take no action.

**Note:** This trading rule is motivated by a simple mean-reversion belief: a large price decline may represent a buying opportunity, while a large price increase may represent an opportunity to realize gains.

**Buy-and-Hold Benchmark:** A buy-and-hold strategy purchases the initial portfolio and makes no further trades. The investor simply holds the original positions throughout the entire simulation horizon.

For comparison, compute the final value of a buy-and-hold portfolio using the same simulated price paths and compare it to the performance of your trading strategy.

### Part E: Portfolio Value

At every time step, compute the value of the trading-strategy portfolio

$$V_t = \text{Cash}_t + \sum_i \text{Shares}_{i,t} \times \text{Price}_{i,t}.$$

Store all portfolio values

$$V_1, V_2, \dots, V_{1000}.$$

Also compute the value of a buy-and-hold portfolio using the same simulated price paths. Recall that a buy-and-hold investor never trades and simply holds the original portfolio throughout the simulation.

At every time step, compute the value of the buy-and-hold portfolio

$$V_t^{BH} = \text{Cash}_t + \sum_i \text{Initial Shares}_i \times \text{Price}_{i,t}.$$

Since the buy-and-hold investor never trades, the cash balance remains unchanged throughout the simulation:

$$\text{Cash}_t = \text{Cash}_0 \quad \forall t.$$

Store all benchmark values

$$V_1^{BH}, V_2^{BH}, \dots, V_{1000}^{BH}.$$

### Part F: Plotting

Create a single plot showing both  $V_t$  and  $V_t^{BH}$  over time. The plot should include:

- A title.
- Clearly labeled axes.
- A legend identifying the trading strategy and the buy-and-hold benchmark.
- A readable scale.

You may use any software package (e.g., R, Python, Excel) to create the plot.

### Part G: Report

Report the following:

- (a) Initial portfolio value.
- (b) Final portfolio value under the trading strategy.
- (c) Final portfolio value under the buy-and-hold benchmark.
- (d) Total number of buy transactions.
- (e) Total number of sell transactions.
- (f) Maximum portfolio value under the trading strategy.
- (g) Minimum portfolio value under the trading strategy.
- (h) Which stock generated the most trades?
- (i) Was the trading strategy profitable?
- (j) Which strategy performed better: the trading strategy or the buy-and-hold benchmark?
- (k) By how much did the better strategy outperform the other?
- (l) Briefly discuss the strengths and weaknesses of the strategy.